

1. Políticas y Control de Acceso

► En el parcial

Esta es la **unidad más evaluada** del segundo parcial: aparece en casi todos los exámenes. Los temas estrella son:

- **Diseño de compartimentos** (construir un retículo a partir de un enunciado en lenguaje natural). Confidencialidad (Bell-LaPadula) e integridad (Biba/Clark-Wilson).
- Identificar la política dado un retículo, y nombrarla bien (BLP vs Biba).
- Resolución de conflictos entre reglas (first-match, best-match, deny-over-allow, grant-all).
- Distinguir **modelo** / **política** / **reglas** de seguridad.
- Preguntas conceptuales V/F (matriz de acceso, ACL vs Capabilities).
- Ejercicios abiertos tipo “sos el CSO”: elegir objetivo de seguridad, modelo, política y reglas.

1.1. Conceptos base

1.1.1. ¿Qué es seguridad? Política y estados

No hay una única definición de seguridad. Igual que en criptografía cada prueba daba una definición distinta de seguridad, al pasar a sistemas dinámicos tampoco hay una definición única. La definición genérica que se usa es:

Definición: Sistema seguro

Si podemos clasificar todos los estados de un sistema en **seguros** e **inseguros**, un sistema es seguro mientras sus estados queden dentro de la política de seguridad (mientras transicione solo por estados que consideramos seguros).

Definición: Política de seguridad

Es la definición de qué estados de un sistema vamos a considerar seguros versus el resto. En un contexto (aplicación, servicio, empresa) la política define los modos y usos que se consideran seguros.

Dos conceptos potentes de esta definición:

1. Necesitamos poder caracterizar, “sacando una foto” del estado actual, si es seguro o no.
2. Un sistema que pasa a un estado inseguro **deja de ser seguro, aunque después vuelva** a un estado seguro. Por eso, ante un servidor comprometido, las buenas prácticas dicen reinstalarlo desde cero: es muy difícil volver a un estado de confianza.

1.1.2. La tríada CIA

- **Confidencialidad:** atañe a la disseminación de información. Hay información accesible a cierto grupo y no a otro.
- **Integridad:** tiene que ver con relaciones de confianza. Garantizar/certificar la calidad de algo (que una transacción ocurre, que los datos son correctos, que el origen es quien dice ser).
- **Disponibilidad:** capacidad de acceder a información o ejecutar una acción en el momento dado. Involucra temporalidad, es dinámica y por eso no se estudia tanto en criptografía (que era estática).

1.1.3. Sujeto, Objeto, Acceso

Definición: Sujeto / Objeto / Acceso

Sujeto: entidad que ejecuta cosas sobre otras entidades; es el origen de un estímulo. Puede ser persona, servicio, aplicación, servidor.

Objeto: destino de esos estímulos. Generalmente recursos: datos, servicios, bases de datos, tablas, programas, archivos.

Acceso: las formas que tienen los sujetos de interactuar con el objeto (leer, escribir, borrar, mover, ejecutar...).

Ejemplo: en Linux los sujetos son usuarios, los objetos son archivos (todo es un archivo) y los accesos son permisos. La tríada se reduce a esta base:

- **Confidencialidad:** existen sujetos que NO pueden acceder a ciertos objetos.
- **Integridad:** existen objetos que solo pueden ser modificados por ciertos sujetos confiables.
- **Disponibilidad:** existen objetos que pueden ser accedidos por ciertos sujetos cuando lo requieran.

Identidad del sujeto: lo que identifica unívocamente al sujeto dentro del universo de posibles sujetos. Es requisito para la autenticación (ej. SID en Windows, UID en `/etc/passwd`).

1.1.4. Modelo de seguridad

Definición: Modelo de seguridad

Modelo formal y lógico que define las reglas e interacciones que permiten implementar un mecanismo de control de acceso. Define qué estados son seguros y cuáles no.

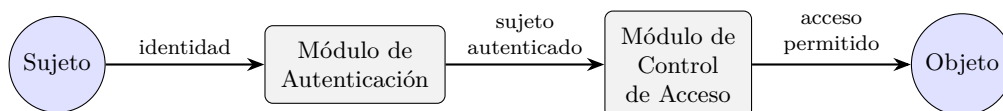
- Los modelos suelen estar **especializados**: hay modelos de confidencialidad, de integridad e incluso de disponibilidad.
- Evitan “reinventar la rueda” (resolver artesanalmente sujeto por sujeto es complejo, propenso a errores y difícil de demostrar).
- **No alcanzan por sí solos en un sistema real**: son la base de muchas implementaciones, pero rara vez se aplican enteramente.

! Importante — remarcado en clase

El profesor remarca mucho la distinción **MODELO / POLÍTICA / REGLAS**:

- **Modelo:** marco formal y lógico (ej. Bell-LaPadula, Biba, Clark-Wilson).
- **Política:** definición de qué estados son seguros para un contexto concreto (ej. “no read up, no write down”).
- **Reglas:** las reglas concretas que implementan la política (ej. las ACL/RuBAC que se cargan).

Un ejercicio típico (2025-2Q) pide identificar explícitamente cada nivel.



*Encadenamiento de módulos (ejercicio de parcial 2013): el sujeto pasa primero por la **autenticación** (¿quién sos?) y luego por el **control de acceso** (¿podés?) antes de llegar al objeto.*

1.2. Modelos de Confidencialidad: Bell-LaPadula (BLP)

Diseñado por Bell y LaPadula (IBM) para el ámbito militar, focalizado en **confidencialidad**. Fue el primer modelo público aprobado para uso en organismos gubernamentales de EE.UU.

(análogo a DES en su rol histórico). Plantea **niveles de seguridad** ordenados; el original usa *Top Secret*, *Secret*, *Confidential*, *Unclassified*, pero admite cualquier secuencia ordenada. Toda la información y todos los sujetos se clasifican en uno de esos niveles.

1.2.1. Las dos reglas (+ la síntesis)

Definición: Condición de Seguridad Simple (ss-property / CSS)

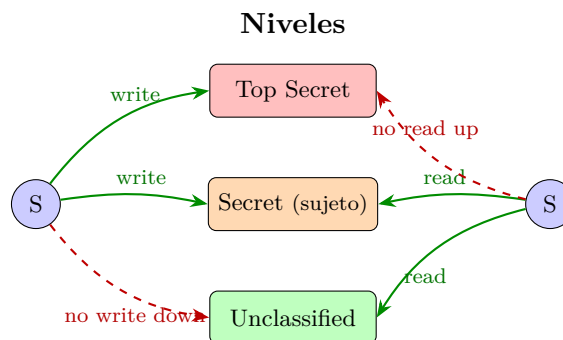
Un sujeto puede **leer** un objeto si y solo si el nivel del sujeto es igual o superior al del objeto. Se traduce como “**no read up**” (leer hacia abajo).

Definición: Regla de cierre / propiedad estrella (*-property)

Un sujeto puede **escribir** un objeto si y solo si el nivel del objeto es igual o superior al del sujeto. Se traduce como “**no write down**” (escribir hacia arriba).

Síntesis (strong star): una persona que necesita leer *y* escribir un objeto solo puede hacerlo con objetos de **su misma clasificación** (por lectura solo accede a su nivel o menores, por escritura solo a su nivel o superiores; la intersección es el mismo nivel).

BLP (confidencialidad): READ DOWN, WRITE UP.



Bell-LaPadula: un sujeto lee hacia abajo (su nivel y menores) y escribe hacia arriba (su nivel y mayores). La intersección lectura $\cap$ escritura es su mismo nivel (strong star).

¿Por qué la *-property? La CSS sola solo resuelve el acceso *directo*. La *-property garantiza **control de flujo de información**: impide que alguien con acceso alto vuelque la información a un nivel más bajo. Ejemplo del editor de texto que guarda una copia temporal: el archivo original queda protegido, pero la copia podría quedar legible para quien no debe. Ejemplo del directivo que “sin querer” filtra información que conoce al hablar con gente de nivel inferior. Caso extremo: en el Proyecto Manhattan se aisló físicamente a los científicos en un pueblo para evitar interacción con niveles inferiores (aplicación de BLP).

! Importante — remarcado en clase

“La idea de este modelo, de alguna manera, es garantizarnos la confidencialidad pero no desde un punto de vista estático: lo que nos garantiza es un control de flujo de información.”

— dicho en clase BLP tiene demostración formal (sobre máquinas de Turing, en el libro de

Bishop) de que no hay ningún camino de información que contravenga sus reglas. Eso es muy fuerte: cuando la información fluye, se copia, se fragmenta y se vuelve a juntar, es muy difícil garantizar la confidencialidad.

Limitación: principio de tranquilidad. BLP asume que la clasificación de sujetos y objetos **no cambia en el tiempo**, lo cual es falso (gente que asciende, información que se hace pública). En la práctica se agregan mecanismos al costado para reclasificar (ej. los avisos “declassified for use” en mails de organismos; se complementa con Clark-Wilson). Es un modelo muy estricto, raro de aplicar completo a todo un sistema; se reserva para las partes más críticas.

1.2.2. Compartimentos y retículo (Lattice)

Solo con 4 niveles el modelo es demasiado simple. Se agrega otra dimensión: además del nivel de seguridad, una **lista de etiquetas** (categorías) asignables a cada sujeto/objeto. Implementa el **need-to-know**: si un general se encarga de criptografía, no tiene por qué conocer los secretos nucleares, sin importar su nivel.

Definición: Compartimento

Un par formado por un **nivel** de seguridad y una **lista de etiquetas**: $(\text{nivel}, \{\text{etiquetas}\})$.

La comparación “nivel superior/inferior” se reemplaza por **dominancia**:

Definición: Dominancia

A **domina** a B ($A \succeq B$) si y solo si se cumplen **ambas** condiciones:

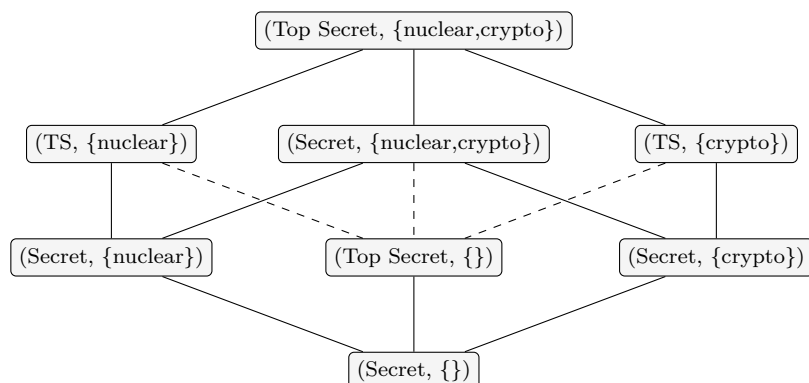
1. $\text{nivel}(A) \geq \text{nivel}(B)$, y
2. $\text{etiquetas}(B) \subseteq \text{etiquetas}(A)$ (las etiquetas de A contienen a las de B).

Reglas con compartimentos:

- Un sujeto puede **leer** un objeto si el compartimento del **sujeto domina** al del objeto.
- Un sujeto puede **escribir** un objeto si el compartimento del **objeto domina** al del sujeto.

Esto forma un **retículo (lattice)**, que es un **conjunto de orden parcial**: no todos los compartimentos son comparables. Ejemplos del libro:

- $(\text{Top Secret}, \{\text{nuclear}\})$ domina a $(\text{Secret}, \{\text{nuclear}\})$.
- $(\text{Secret}, \{\text{nuclear}\})$ domina a $(\text{Secret}, \{\})$.
- $(\text{Top Secret}, \{\text{nuclear}, \text{crypto}\})$ domina a $(\text{Top Secret}, \{\text{nuclear}\})$.
- $(\text{Secret}, \{\text{nuclear}\})$ y $(\text{Secret}, \{\text{crypto}\})$ **NO** tienen relación: mismo nivel, pero ninguna lista de etiquetas es subconjunto de la otra. No hay dominancia en ningún sentido $\Rightarrow$ están efectivamente separados (ni lectura ni escritura entre ellos).



*Retículo (diagrama de Hasse) del libro: cada arista sube hacia el que **domina**. $(\text{Secret}, \{\text{nuclear}\})$ y $(\text{Secret}, \{\text{crypto}\})$ **no son comparables** (ninguna lista de etiquetas contiene a la otra) $\Rightarrow$ están separados.*

A nivel implementación es simple: a cada objeto y a cada sujeto se le almacena como metadata un nivel y una lista de etiquetas; resolver cada operación con las dos reglas es directo.

△ Cuidado — error típico

El error más penalizado de la unidad: **confundir las direcciones de lectura/escritura**. BLP (confidencialidad) es **read down / write up**; Biba (integridad) es lo opuesto. No alcanza con saber “arriba/abajo”: hay que decir bien si es confidencialidad o integridad.

1.3. Modelos de Integridad: Biba y Clark-Wilson

1.3.1. Modelo Biba

Creado por Kenneth J. Biba. Fue el primer modelo público que **no buscaba confidencialidad sino proteger la integridad**. Es en realidad una familia de ~ 7 modelos. Define **niveles de integridad** (calidad/confiabilidad de la información). Una clasificación alternativa que el profesor prefiere: *información no confiable*, *poco confiable*, *confiable*, *muy confiable*, *fidedigna*. Importante en sistemas documentales / de evidencia: contabilidad histórica, registros financieros que hay que retener, sistemas judiciales, medios de prensa, declaraciones fiscales.

Biba estricto. Es Bell-LaPadula al revés:

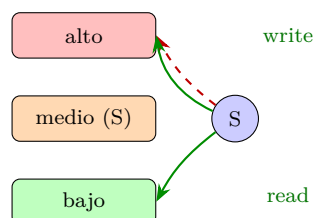
Definición: Reglas de Biba estricto

Lectura: un sujeto puede leer un objeto si y solo si el nivel de integridad del **objeto** es igual o superior al del sujeto $\Rightarrow$ “**no read down**”.

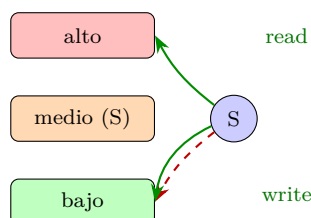
Escritura: un sujeto puede escribir un objeto si y solo si el nivel de integridad del **objeto** es igual o menor al del sujeto $\Rightarrow$ “**no write up**”.

Biba (integridad): READ UP, WRITE DOWN. (Invertir las flechas de un retículo BLP da Biba.)

BLP (confidencialidad)



Biba (integridad)



Biba es el espejo de BLP: las flechas de lectura y escritura están invertidas. Confidencialidad lee abajo / escribe arriba; integridad lee arriba / escribe abajo.

Intuición: “uno es tan confiable como las fuentes que utiliza”. Para considerar a un sujeto confiable, la información en la que se basa debe ser de igual o mayor confianza (no puede leer información dudosa). Y lo que un sujeto genera/escrbe tiene a lo sumo su mismo nivel, nunca mayor. Ejemplo: el SO te advierte al ejecutar un archivo bajado de Internet (origen poco confiable) y no si lo bajaste del App Store.

Biba Low-Watermark. Variante **dinámica** (vs. las estáticas). No restringe la lectura: cualquier sujeto puede leer cualquier objeto, pero al hacerlo **el nivel de integridad del sujeto se reclasifica al mínimo** entre el que tenía y el del objeto. Si interactúa con algo de baja integridad, su nivel baja automáticamente, y deberá revalidarse para recuperar el nivel anterior. Base de muchos sistemas “adaptativos”.

► En el parcial

Biba Low-Watermark fue tomado en el Recuperatorio 2025. Recordá: *lectura libre, pero el sujeto baja su nivel al del objeto leído (toma el mínimo)*.

1.3.2. Modelo Clark-Wilson

Creado por David D. Clark y David R. Wilson. Modelo de **integridad** más acotado, pero introduce conceptos clave. Opera en un escenario con información y sujetos confiables y no confiables al mismo tiempo:

- Si sujeto e información son no confiables, que hagan lo que quieran.
- Un sujeto confiable no puede usar información no confiable; información confiable no puede ser usada por un sujeto no confiable.

¿Qué pasa cuando un sujeto confiable quiere usar información no confiable? Entra el mecanismo:

- Los datos pueden ser **restringidos/protegidos (CDI, Constrained Data Item)** o **sin restricción (UDI, Unconstrained Data Item)**.
- Los datos restringidos solo se modifican mediante un **Proceso de Transformación (TP)**.
- Hay un **Proceso de Verificación de Integridad (IVP)** independiente que valida.
- **Separation of duty (separación de tareas/privilegios)**: quien modifica los datos no puede ser el mismo que verifica la integridad.

Reglas de certificación y enforcement (del modelo formal): C1 IVP valida estado de CDI; C2 los TP preservan estado válido; C3 separación de tareas adecuada; C4 los TP escriben a log; C5 los TP validan los UDI. E1 los CDI se cambian solo por TP autorizados; E2 usuarios autorizados para TP; E3 usuarios autenticados; E4 las listas de autorización las cambia solo el security officer.

! Importante — remarcado en clase

“La idea de este modelo de arbitración de Clark-Wilson es la base por la cual existen los kernels del sistema operativo y existe el concepto de User Space / Kernel Space como cosas separadas.”

— dicho en clase Ejemplo bancario: para mover montos altos hacen falta dos personas (o procesos independientes), una verifica las firmas y otra ejecuta la transacción. En el SO, un *syscall* arma datos en user space, un proceso los transforma/mueve a kernel space, otro proceso verifica su integridad y recién ahí se ejecuta en el espacio protegido.

1.3.3. Más allá de los modelos teóricos

Son modelos teóricos, difíciles de aplicar enteros a un sistema. No esperes encontrar el nombre explícito, pero aparecen partes:

- Compartimentos de BLP → sistemas que manejan **información médica** (regulatorios).
- Biba → sistemas de clasificación y archivo para **medios de prensa, legajos judiciales, declaraciones fiscales**.
- Clark-Wilson → concepto **kernel space / user space**.

1.4. Control de Acceso: clasificación

Definición: Control de acceso

Mecanismo de seguridad que regula quién puede acceder a qué recursos de la aplicación y bajo qué condiciones.

Dos grandes familias:

Definición: Mandatory Access Control (MAC)

Control que se aplica en forma **obligatoria** a todos los accesos de los sujetos a los objetos. Existen reglas universales que se aplican a todos. Los modelos vistos (BLP, Biba) son MAC. Se aplica sobre: todos los accesos, los cambios en las reglas/atributos, y el intercambio de información entre sujetos.

Definición: Discretionary Access Control (DAC)

La asignación y modificación de permisos queda a **discreción** del sujeto responsable del objeto o de un administrador. Las propiedades de seguridad las da el administrador según cómo configure, no el sistema. Típico de repositorios donde los sujetos crean los objetos (file systems, object storage).

MAC es muy rígido; para sistemas de propósito general (SO, bases de datos, suites de oficina) los controles tienden a ser **discrecionales** por flexibilidad.

1.4.1. Control discrecional por extensión

Definir el conjunto de accesos *enumerándolos* uno por uno.

Matriz de acceso. Matriz centralizada: sujetos (filas) $\times$ objetos (columnas); en cada celda, las acciones permitidas para esa combinación. Simple de entender, pero con problemas: escalabilidad (crece violentamente), eficiencia de búsqueda, mantenimiento propenso a errores, espacio, y mala adaptación a entornos dinámicos. Para una computadora una matriz enorme no es problema; **el cuello de botella es el humano** que debe cargar los valores a discreción.

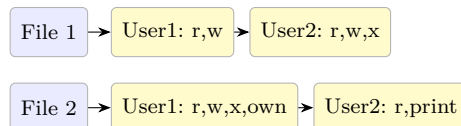
△ Cuidado — error típico

Falso clásico de parcial: “**una matriz de acceso bien diseñada evita el flujo de información no deseado**”. **FALSO**. La matriz (DAC, por extensión) solo controla el acceso *directo*; no garantiza control de flujo de información. Para eso hacen falta modelos MAC tipo Bell-LaPadula (la *-property es justamente lo que controla el flujo).

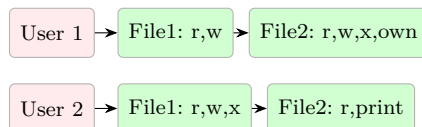
		Objetos→	
		File 1	File 2
Sujetos	User 1	r, w	r, w, x, own
	User 2	r, w, x	r, print

Matriz de acceso: sujetos (filas) $\times$ objetos (columnas); cada celda lista las acciones permitidas (basado en la slide “Control por extensión”).

ACL (por columna = objeto)



Capabilities (por fila = sujeto)



*Descomposición de la matriz: **ACL** recorre por columnas (a cada objeto, quién puede); **Capabilities** recorre por filas (a cada sujeto, a qué puede). Misma información, distinta forma de almacenarla.*

ACL (Access Control List). Como las matrices reales son **dispersas** (permisos por “islas”), se representan de forma compacta: para cada objeto, la lista de pares (sujeto, acción) permitidos. Es la misma información que la matriz, pero compacta. Se complementa con **permisos por defecto**, **grupos/agrupaciones** y **patrones** (/users/*, /home/**/*.*tmp).

1.4.2. Control discrecional por comprensión

Definir el conjunto de accesos por sus **propiedades**, mediante **reglas** más compactas que reconstruyen la matriz bajo demanda. Tres modelos según criterio de agrupación:

- **RBAC (por roles)**: se estereotipan **roles** (Account Manager, DBA, Gestor de pacientes, Analista de compras...) que agrupan permisos; los roles se asignan a sujetos o grupos. Tras definir los roles, la gestión diaria es simple. (En sistemas tradicionales un usuario tiene un solo rol; en los complejos puede tener varios, lo que genera conflictos.) Algunos libros lo llaman MAC; en realidad la asignación de roles suele ser discrecional.
- **ABAC (por atributos)**.
- **RuBAC (por reglas)**: no asigna sujetos como los roles; usa una **lista ordenada de reglas** (Principal, Objeto, Tipo de Acceso, Permitido/Bloqueado) y aplica una política de resolución.

△ Cuidado — error típico

Para usuarios **anónimos** conviene **ACL sobre Capabilities (CAP)** (parcial 2018). Las capabilities asocian permisos al sujeto (necesita identidad/token); con usuarios anónimos no hay a quién asociar la capability, así que conviene listar en el objeto quién puede (ACL).

1.5. Resolución de conflictos entre reglas

Cuando varias reglas aplican de forma contradictoria (o cuando se combinan varios mecanismos de control sobre la misma interacción), hay que resolver el conflicto. Es **crucial** entender cómo resuelve cada sistema, porque *el mismo juego de reglas da resultados distintos según la política de resolución*.

- **First match**: las reglas tienen orden; se aplica la **primera** que hace match.
- **Best match**: aplica la regla **más específica** (antes que las generales).
- **Deny over Allow**: cualquier regla que deniegue explícitamente tiene precedencia sobre las que permiten (equivale a: si hay múltiples reglas que aplican, *todas* deben permitir; si no, se deniega).
- **Grant-all / augment / override**: políticas adicionales (grant-all $\approx$ permitir si alguna regla permite).

Ejemplo (firewall). Reglas ordenadas; conexión $10,1,1,1 \Rightarrow 20,1,1,1:80$. Matchean la regla específica ($10.1.1.1 \dots \text{Accept}$) y la general ($10.1.1.0/24 \dots \text{Deny}$).

- **First match:** gana la primera $\Rightarrow$ **Accept**.
- **Best match:** gana la más específica $\Rightarrow$ depende del caso (para $10,3,3,9 \Rightarrow 20,3,3,9:80$, la específica **Deny** gana sobre $10.3.3.0/24$ **Accept**).
- **Deny over Allow:** como hay una regla que deniega, **se frena** $\Rightarrow$ **Deny**.

En first-match las reglas dicen “no permitas esta red salvo desde tal servidor”; en best-match “permití toda la red salvo tal IP”. La forma de escribir las reglas cambia según el motor.

► En el parcial

Ejercicio recurrente (2015, 2016, Recup 2023): dado $ACL(o) = \{(Manuel, r), (Directivos, w), (*, w)\}$, donde Manuel *es* Directivo, decidir bajo qué política puede **leer** y **escribir**.

- **Leer (r):** solo la regla $(Manuel, r)$ otorga lectura. Manuel **puede leer** bajo cualquier política que considere esa regla (first-rule si está primero, grant-all, augment).
- **Escribir (w):** las reglas $(Directivos, w)$ y $(*, w)$ otorgan escritura y Manuel encaja en ambas; ninguna regla la deniega, así que **puede escribir**.
- Bajo **grant-all:** alcanza con que una regla lo permita $\Rightarrow$ Manuel puede leer Y escribir.
- Razonar siempre: qué reglas matchean al sujeto, qué accede cada una, y cómo combina la política elegida (first-rule, grant-all, override, augment).

1.5.1. Reglas basadas en contexto

El **contexto** son los datos asociados al sujeto, objeto, momento o lugar del acceso, que cambian independientemente de las reglas: momento del día, ubicación geográfica del origen, departamento del que accede, si el usuario estaba de vacaciones, nivel de confiabilidad de la máquina. Permiten detectar anomalías:

- Login desde China y desde Argentina con 5 min de diferencia.
- Usaria de licencia por maternidad que se loguea a una base de datos sensible.
- Usuario que abre dos VPN en menos de 5 min con dos IP distintas.

Su nicho son los **sistemas de prevención de fraude**. (Un fraude es un abuso *dentro* de las reglas del sistema; un problema de seguridad es un abuso *salteándose* las reglas.) Son complejos; en el modelo **Zero-Trust** se define una entidad externa (**Policy Engine**) que evalúa todas las condiciones de contexto.

1.5.2. Open Policy Agent (OPA)

Framework estándar que unifica, mediante un lenguaje programable, la implementación de políticas de decisión, **desacoplando** la política del sistema de control. El servicio delega cada decisión a OPA: le envía una *query* (JSON) y recibe una *decision* (JSON). OPA evalúa una **política (Rego)** contra **datos contextuales (JSON)**. Las políticas se escriben en el lenguaje **Rego** (ej. en Kubernetes, denegar despliegues de imágenes que no vengan de un dominio dado).

1.6. Ejemplos reales de control de acceso

- **Unix/Linux file system:** “engañosamente simple”. Permisos **rw**x en 3 secciones (usuario / grupo / otros). Tipo de archivo: **d** directorio, **-** archivo regular, **l** symlink. Al crear, un archivo hereda atributos del directorio; **umask** define una máscara que quita permisos a los archivos nuevos. Es **DAC**.
- **SELinux** (Security Enhanced): control **adicional al DAC**, basado en reglas que responden “¿puede este sujeto acceder a este objeto?”. Procesos y recursos tienen un *SELinux context* (ej. `/var/tmp: t_var_tmp`, Port 80: `t_port_www`). Usa macros que expanden a conjuntos de

reglas. Permite que dos procesos con UID 0 (root) tengan restringido el acceso a los recursos del otro. Se usa en *hardening* de servidores.

- **Windows file system:** DAC con dueño; usuarios y grupos; nivel básico y avanzado (más granular, ~ 11–13 operaciones). **Herencia** de permisos del contenedor, que *persiste a la creación* (a diferencia de Unix). Calcula **permisos efectivos**; **Deny tiene precedencia sobre Allow**; reglas locales sobre heredadas. Cada entrada puede Allow / Deny / dejar en blanco (en blanco = respeta el heredado).
- **PostgreSQL:** modelo basado en **roles** (un rol = usuario, grupo u otro rol; propiedad LOGIN). Todo objeto creado por un rol tiene a ese rol como **owner** (DAC). **Policies** (Row Level Security) permiten need-to-know a nivel de registros (ej. cada vendedor ve solo sus clientes vía `salesperson_id = current_user`).
- **Squid Web Proxy:** proxy web que controla por dónde navegan los usuarios. Sistema de reglas (mal llamado ACL) de tipo **MAC**; reglas ordenadas que filtran por dominio, protocolo, URL, headers, **listas negras (blacklists)**. En la facultad bloquea sitios de streaming con esta tecnología.
- **AWS:** combina dos mecanismos: **políticas asociadas al Principal (MAC)** (definidas para la cuenta, por administradores) y **políticas asociadas al Recurso (DAC)** (las define quien crea el recurso, ej. S3 bucket policy).
- **API Gateway:** capa delante de las APIs; control basado en reglas (MAC) por URL, dominio, headers, method; y por contexto (req/seg máximos, ancho de banda).

1.6.1. OAuth 2.0 (Open Authorization)

Estándar de **autorización** para que aplicaciones accedan a recursos en nombre del usuario **sin compartir sus credenciales**. Funciona emitiendo **tokens de acceso temporales y específicos**. Es el “entrar con tu cuenta de Google”.

Flujo: el usuario (Resource Owner) accede a la aplicación; la app lo redirige al **Authorization Server**, que lo autentica y le muestra la pantalla de consentimiento; al aceptar, el servidor envía un **authorization code** a la app, que lo intercambia por un **access token**; con ese token la app pide el recurso al **Resource Server**.

- **Access token** (vida corta: una hora, días) + **refresh token** (uso único; al usarlo devuelve un access y un refresh nuevos). Mecanismo para recuperarse ante robo de tokens y sostener la “sesión infinita”.
- **Scope (DAC):** permisos específicos que la app solicita (ver email, postear, leer calendario...). Se muestran en la pantalla de consentimiento; el token se limita a los scopes otorgados. Cada servicio verifica que el token sea válido y contenga su scope. Quien determina el control de acceso es **el dueño del recurso**.

1.7. Metodología: diseño de compartimentos (ejercicio estrella)

► En el parcial

Es el ejercicio: dados sujetos, objetos y requisitos en **lenguaje natural**, construir un retículo de compartimentos. Casos históricos:

- **Confidencialidad:** restaurante Michelin (parciales 2015 y 2016) ⇒ Bell-LaPadula.
- **Integridad:** clínica de cirugías estéticas (2013) y caso deepfake ⇒ Biba.

SIEMPRE hay que justificar requisito por requisito y nombrar bien la política.

Pasos.

1. **Identificar el objetivo de seguridad:** ¿el enunciado pide *ocultar* información (confidencialidad ⇒ BLP) o *proteger la calidad/origen* de la información (integridad ⇒ Biba / Clark-Wilson)? El deepfake/integridad: “Necesitan estructurarse con un modelo similar a Biba”.

2. **Listar sujetos y objetos** del enunciado.
3. **Definir los niveles** (jerarquía vertical) y las **etiquetas/categorías** (need-to-know horizontal: áreas, proyectos, temas que deben ir separados).
4. **Asignar a cada sujeto y objeto su compartimento** (nivel, {etiquetas}), justificando con el requisito que lo motiva.
5. **Dibujar el retículo** con la relación de dominancia (orden parcial): aristas hacia el que domina; compartimentos sin etiquetas comparables quedan separados.
6. **Verificar cada requisito** contra las reglas del modelo (read down/write up para BLP; read up/write down para Biba), uno por uno.
7. **Nombrar la política** explícitamente (BLP “no read up, no write down” / Biba “no read down, no write up”).

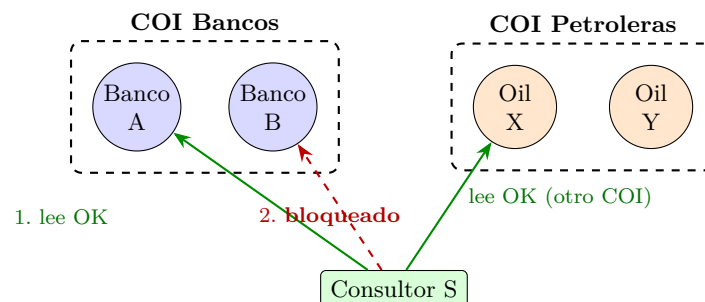
► En el parcial

Identificar la política dado un retículo (parcial 2018, jerarquía CEO → Gerentes → Empleado):

- Mirar las flechas/dirección de los flujos permitidos. Si el flujo de *lectura* va hacia abajo (un superior lee lo de abajo) y la *escritura* hacia arriba ⇒ **Bell-LaPadula** (confidencialidad).
- Si están **invertidas** (lectura hacia arriba, escritura hacia abajo) ⇒ **Biba** (integridad). Invertir las flechas de un retículo BLP da Biba.

1.7.1. Chinese Wall / Muralla China

Modelo de **conflicto de interés**. Se agrupan los objetos en **clases de conflicto de interés** (COI / **Conflict of Interest**); dentro de cada clase hay **datasets de compañías** (CD). La regla: un sujeto que accedió a datos de una compañía dentro de una clase de COI **no puede acceder a los datos de otra compañía competidora** de la misma clase (ya consultó a un competidor). Tomado en el parcial 2018.



*Chinese Wall: tras leer Banco A, el consultor **no puede** leer a Banco B (competidor del mismo COI), pero sí una petrolera de otro COI.*

1.8. Mentalidad de seguridad (clase de consulta)

! Importante — remarcado en clase

De la clase de consulta, el profesor remarcó:

- Los ejercicios son **situaciones laborales** donde uno *decanta* la política general a partir del enunciado.
- Hay que “**activar el chip paranoico**”.
- **Defensa en profundidad = encadenar controles** (varias capas).
- Para el caso deepfake/integridad: “*Necesitan estructurarse con un modelo similar a Biba*”.

► En el parcial

Ejercicio abierto tipo “sos el CSO”: pasos para responder.

1. **Identificar el objetivo de seguridad:** confidencialidad, integridad o disponibilidad (puede haber más de uno).
2. **Elegir el modelo** acorde (BLP para confidencialidad; Biba/Clark-Wilson para integridad).
3. **Definir la política** concreta (qué estados son seguros).
4. **Bajar a reglas** (ACL, RuBAC, compartimentos, MAC/DAC, resolución de conflictos) y, si conviene, encadenar controles (defensa en profundidad).